



ARA0110 PADRÕES DE PROJETO

Aula 11: SOLID





CONCEITOS





HISTÓRICO

A programação orientada a objetos surgiu com a promessa de resolver o caos do código procedural através do encapsulamento e da reutilização. Porém, sem diretrizes rígidas, boa parte dos desenvolvedores criaram problemas estruturais novos, fazendo com que a iniciativa voltasse a estaca zero. Robert C. Martin observou que a falta de padrões gerava quatro sintomas de causavam mau cheiro de design (design smells).



DESIGN SMELLS

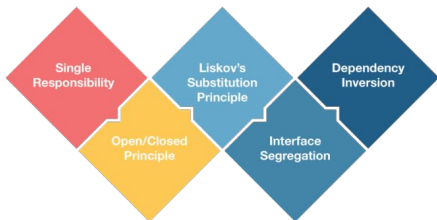
- **Rigidez:** O sistema é difícil de mudar, onde uma alteração simples exige uma reação em cadeia de modificações em múltiplos módulos do sistema
- **Fragilidade:** Ao quebrar ou alterar o código em um lugar, partes totalmente desalinhadas e distantes do sistema param de funcionar sem explicação
- **Imobilidade:** Contrariando a ideia de reutilização, reaproveitar um módulo em outro sistema se torna impossível uma vez que ele está rigidamente acoplado ao sistema atual
- **Viscosidade:** Fazer a coisa certa é muito mais difícil do que fazer uma gambiarra. Isso faz o desenvolvedor ser induzido a quebrar o design em favor de “entregar rápido”



HISTÓRICO

Conversando com outros desenvolvedores, em fóruns, Martin discutia formas de estruturar classes de forma eficiente. Durante esse período, ele identificou os gargalos por dependências mal gerenciadas e compilou os primeiros rascunhos de seus artigos de design de software.

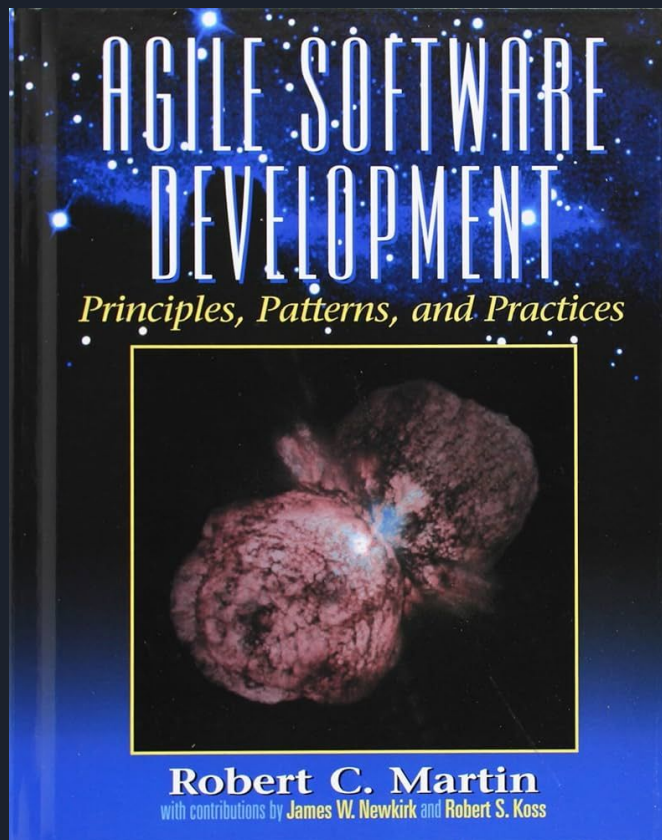
S.O.L.I.D.



PUBLICAÇÃO

Em 2000, ele publicou o famoso compilado **Design Principles and Design Patterns**, onde os 5 princípios foram formalmente apresentados juntos. Em 2004, Michael Feathers propôs organizar as iniciais de cada um dos princípios, surgindo assim o acrônimo **SOLID**. Ele se trata de um conjunto de princípios de boas práticas para design de sistemas orientados a objetos, com o intuito de trazer robustez e eficiência no desenvolvimento orientado a objetos.

LIVRO



SOLID

S

Princípio da
Responsabilidade Única

O

Princípio
Aberto/Fechado

L

Princípio da
Substituição de Liskov

I

Princípio da Segregação
de Interface

D

Princípio da Inversão de
Dependência



PROBLEMAS PARA O SOLID

- **Classes gigantes:** Milhares de linhas, muitas responsabilidades, tudo misturado
 - **Alto acoplamento:** Código extremamente dependente de implementações
 - **Alterações quebrando funcionalidades:** Fragilidade no design das classes
 - **Dificuldade de testar:** Testar a aplicação é um desafio que muitas vezes desencoraja a atividade
 - **Reaproveitamento baixo:** Pouca possibilidade de reutilizar componentes em outros projetos
 - **Crescimento desorganizado do sistema:** A aplicação acaba se tornando uma quimera
- 



OBJETIVOS COM O SOLID

- **Código mais modular:** Separação clara e organizada de componentes e suas responsabilidades
 - **Código mais flexível:** Facilidade em fazer adaptações inerentes a cada projeto
 - **Código mais reutilizável:** Mais possibilidades de aproveitar componentes entre projetos
 - **Código menos acoplado:** Classes menos dependentes de implementações, facilitando flexibilidade e testabilidade
 - **Código mais fácil de manter:** Fazer modificações não se torna mais algo problemático
- 

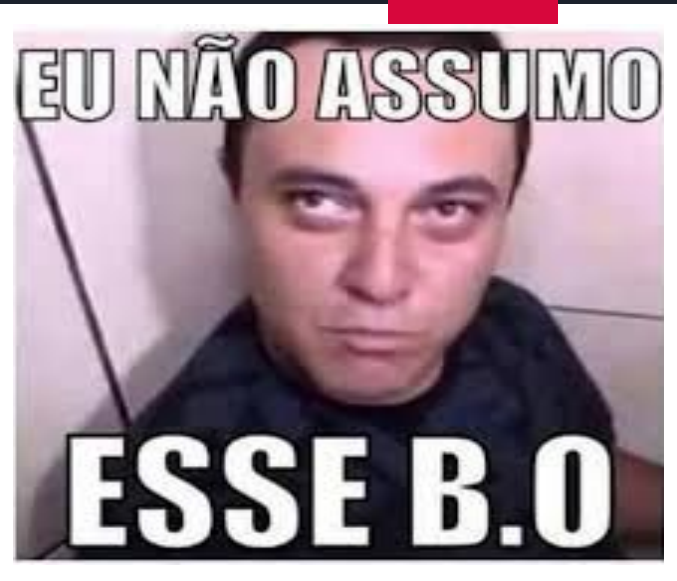
EXEMPLO

```
public class RelatorioService {  
  
    public void gerarRelatorio() {  
        // lógica  
    }  
  
    public void salvarBanco() {  
        // persistência  
    }  
  
    public void enviarEmail() {  
        // envio de email  
    }  
  
    public void imprimir() {  
        // impressão  
    }  
}
```

Quais problemas podem surgir se essa classe crescer?

PRINCÍPIOS





RESPONSABILIDADE ÚNICA

Conceito: Uma classe deve ter um, e apenas um, motivo para mudar.

A ideia do princípio é termos uma separação clara entre o que cada classe deve fazer dentro de uma aplicação. Por exemplo, Devemos separar a lógica de negócios da lógica de banco de dados ou de exibição.



CENÁRIO

Quando uma única classe executa múltiplas tarefas distintas, como gerenciar dados em um modelo, acessar o banco de dados e enviar e-mail, ocorrem problemas graves:

- Alto acoplamento
- Dificuldade de manutenção
- Conflitos de mesclagem



PROBLEMAS

- **Alto acoplamento:** Alterações em uma regra de negócio quebram funcionalidades vizinhas sem qualquer relação direta
- **Dificuldade de manutenção:** O código se torna confuso e difícil de testar de forma isolada
- **Conflitos de mesclagem:** Diferentes desenvolvedores alteram o mesmo arquivo simultaneamente por motivos totalmente distintos



EXEMPLO

```
public class Funcionario {  
    private String nome;  
    private double salarioBruto;  
  
    public void calcularSalario() {  
        // Calcular o salário do funcionário  
    }  
  
    public void gerarRelatorio() {  
        // Gerar relatório sobre os dados  
        // do funcionário  
    }  
  
    public void salvarBanco() {  
        // Salvar os dados do funcionário  
    }  
}
```

Quais problemas podem ser identificados nessa classe?





PROBLEMA

O problema dessa classe gira em torno dela executar várias responsabilidades: calcular o salário líquido, gerar um relatório sobre os dados e persistir as informações em base de dados. Essa mistura de lógicas na mesma classe fere o princípio, e deve ser evitado.

EXEMPLO

```
public class Funcionario {  
    private String nome;  
    private double salarioBruto;  
  
    public String getNome() {  
        return this.nome;  
    }  
  
    public double getSalarioBruto() {  
        return this.salarioBruto;  
    }  
}
```

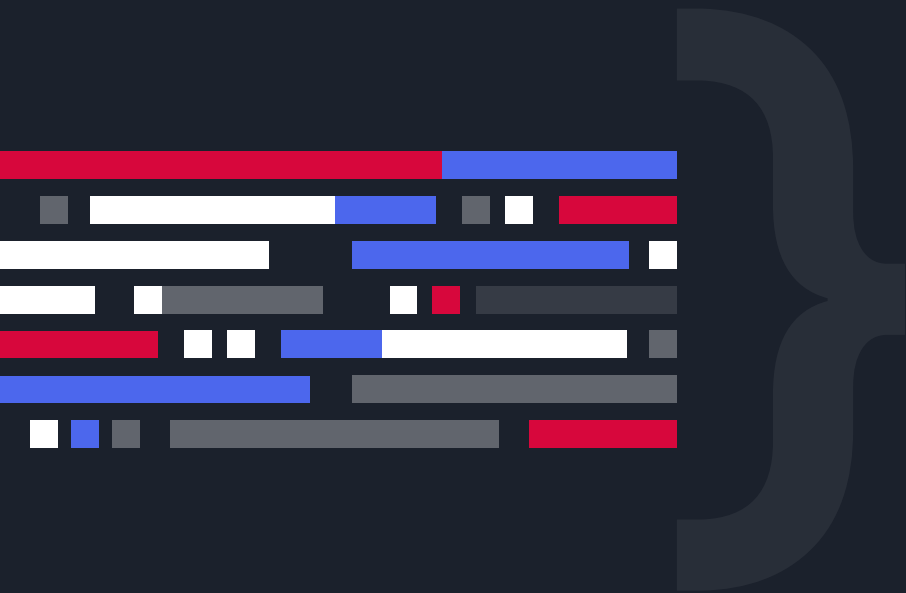
```
public class FolhaPagamentoService {  
  
    public double calcularSalario(Funcionario funcionario) {  
        return funcionario.getSalarioBruto();  
    }  
}
```



EXEMPLO

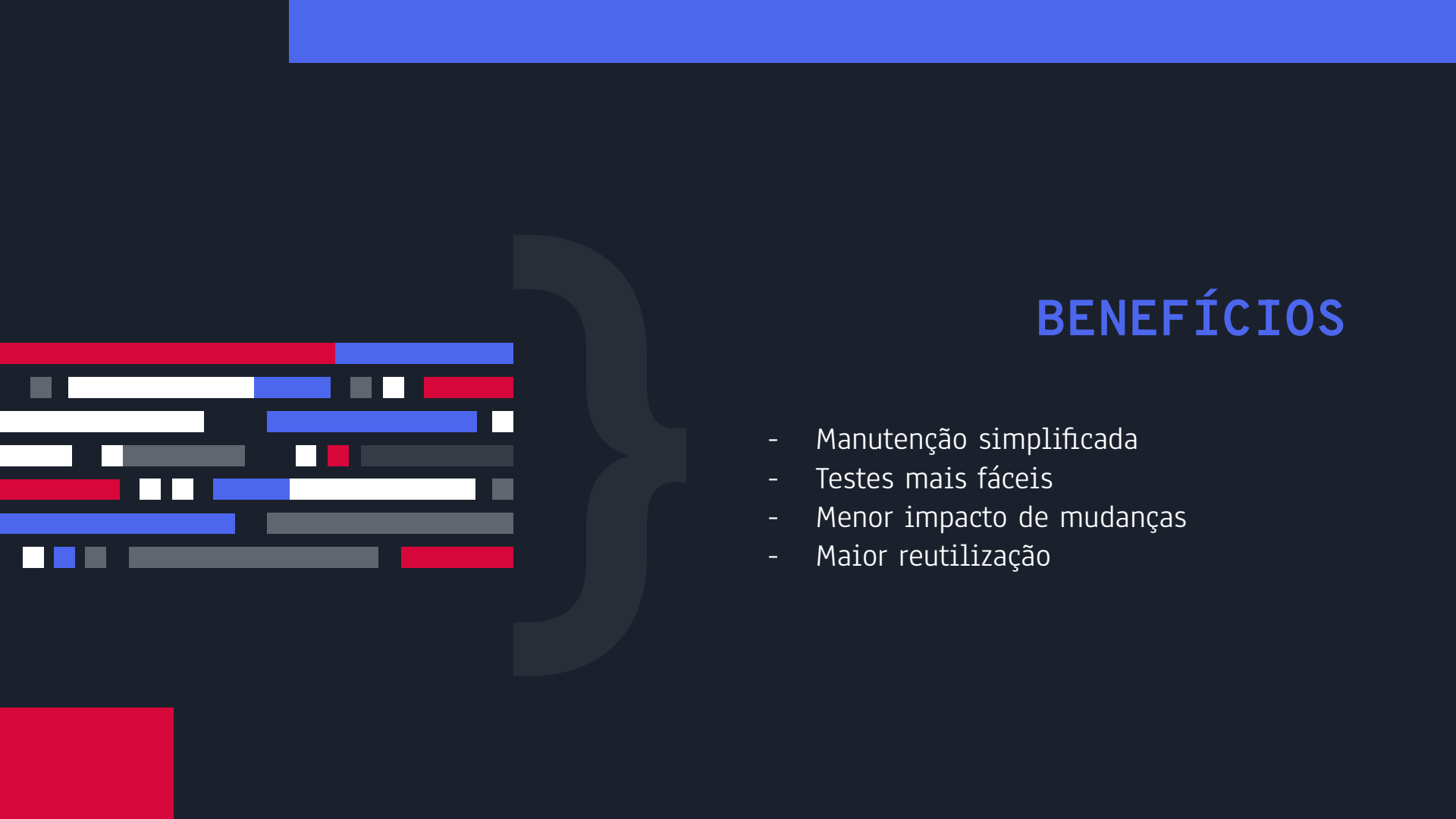
```
public class FuncionariosRepository {  
  
    private Database db;  
  
    public void salvar(Funcionario funcionario) {  
        this.db.save(funcionario);  
    }  
}  
  
public class RelatorioFuncionario {  
  
    public void gerar(Funcionario funcionario) {  
        System.out.println(funcionario.getNome());  
        System.out.println(funcionario.getSalarioBruto());  
    }  
}
```





SOLUÇÃO

Agora, temos classes que descrevem responsabilidades bem definidas: uma classe que armazena os dados do funcionário, uma classe de serviço para regras de negócio, uma classe de persistência em base de dados e uma classe para geração de relatório. As classes possuem independência para expandir conforme a necessidade.



BENEFÍCIOS

- Manutenção simplificada
- Testes mais fáceis
- Menor impacto de mudanças
- Maior reutilização



HORA DE EXERCÍCIO

Considere o exemplo a seguir de uma classe que representa um pedido em um sistema de vendas. Analise a sua estrutura e proponha modificações que façam com que a classe passe a adotar o princípio da Responsabilidade Única.

EXERCÍCIO

```
public class Pedido {  
  
    public void calcularFrete() {}  
  
    public void salvarPedido() {}  
  
    public void enviarEmailConfirmacao() {}  
  
    public void emitirNotaFiscal() {}  
  
}
```



ABERTO/FECHADO

Conceito: Uma entidade deve estar **aberta** para extensão e **fechada** para modificação.

A ideia do princípio é conseguirmos alterar o comportamento de um componente adicionando código novo, e nunca alterando o código original que já funciona.



CENÁRIO

Quando novos recursos exigem a alteração constante de classes existentes, o sistema manifesta sintomas graves:

- Efeito cascata
- Complexidade condicional
- Testes repetitivos

PROBLEMAS

- **Efeito cascata:** Mexer em um código consolidado introduz novos bugs em funcionalidades antigas
- **Complexidade condicional:** O código se enche de estruturas **if/else** ou **switch** difíceis de ler
- **Testes repetitivos:** Lógicas antigas precisam ser testadas novamente a cada nova implementação de negócio

EXEMPLO

```
public class CalculadoraDesconto {  
  
    public double calcular(String tipoCliente, double valor) {  
        if(tipoCliente.equals("COMUM")) {  
            return valor * 0.05;  
        }  
  
        if(tipoCliente.equals("VIP")) {  
            return valor * 0.10;  
        }  
  
        return 0;  
    }  
}
```

Quais problemas
podem ser
identificados nessa
classe?





PROBLEMA

Neste exemplo, usamos estruturas condicionais para determinar comportamentos diferentes. Caso uma nova opção de desconto entre em vigor, precisaremos modificar a classe para adicionar uma nova verificação **if/else** dentro do código, ferindo assim o princípio.

EXEMPLO

```
public interface Desconto {  
    double calcular(double valor);  
}  
  
public class DescontoComum implements Desconto {  
  
    public double calcular(double valor) {  
        return valor * 0.05;  
    }  
}  
  
public class DescontoVip implements Desconto {  
  
    public double calcular(double valor) {  
        return valor * 0.10;  
    }  
}
```

```
public class CalculadoraDesconto {  
  
    public double calcular(Desconto desconto, double valor)  
    {  
        return desconto.calcular(valor);  
    }  
}
```





SOLUÇÃO

Usamos uma interface para padronizar a operação de calcular um desconto. Em seguida, implementamos as classes que conterão a lógica de cálculo de desconto para cada cenário, e repassamos a operação para a calculadora. Dessa forma, para adicionar um novo cenário de desconto, basta implementar uma nova classe que obedeça a mesma interface.



BENEFÍCIOS

- Extensibilidade
- Menos risco de quebrar código
- Melhor manutenção
- Sem utilização de condicionais dentro da lógica de negócio



HORA DE EXERCÍCIO

Crie um sistema de frete para envio de compras on-line. Os fretes poderão ser feitos via **Sedex**, **PAC** ou **transportadora**, onde cada modalidade possui suas regras, datas possíveis de entrega e valores aplicados ao frete. Implemente essa dinâmica sem o uso de estruturas condicionais na lógica principal.



SUBSTITUIÇÃO DE LISKOV

Conceito: Objetos derivados devem poder substituir objetos de classe base sem quebrar o comportamento esperado.

Aqui, uma classe não deve apenas herdar os métodos da classe ancestral, mas também respeitar rigorosamente o comportamento esperado por quem consome a classe base.



CENÁRIO

Quando uma subclasse quebra as expectativas da classe ancestral, o design de software falha de formas perigosas:

- Exceções inesperadas
- Gambiarras no código
- Comportamento imprevisível

PROBLEMAS

- **Exceções inesperadas:** Métodos herdados lançam erros de “Não implementado” por não fazerem sentido na subclasse
- **Gambiarrras no código:** O sistema se enche de verificações de tipos (**instanceof**, **typeof** etc) para evitar erros
- **Comportamento imprevisível:** O programa passa a se comportar de forma errada dependendo do objeto recebido

EXEMPLO

```
public class Ave {  
  
    public void comer() {  
        System.out.println("Alpiste? Isso é comida pra passarinho!");  
    }  
  
    public void voar() {  
        System.out.println("Voando fortemente...");  
    }  
}  
  
public class Pinguim extends Ave {  
  
    @Override  
    public void voar() {  
        throw new UnsupportedOperationException("Pinguim não voa.");  
    }  
}
```

Quais problemas
podem ser
identificados nessa
classe?



PROBLEMA

Neste cenário, temos uma situação em que a utilização de um objeto derivado específico causa inconsistências no sistema. É esperado que uma ave seja capaz de voar, mas um pinguim é uma ave que não voa. Utilizar a herança dessa forma fere o princípio, e devemos contornar isso.

EXEMPLO

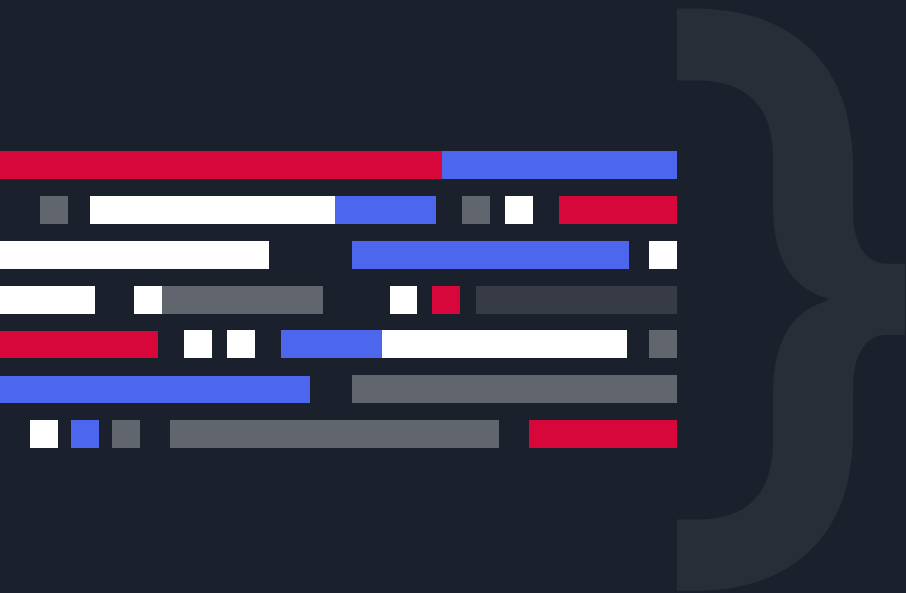
```
public abstract class Ave {  
  
    public void comer() {  
        System.out.println("Alpiste? Isso é comida pra passarinho!");  
    }  
}  
  
public interface AveVoadora {  
  
    void voar();  
  
}
```



EXEMPLO

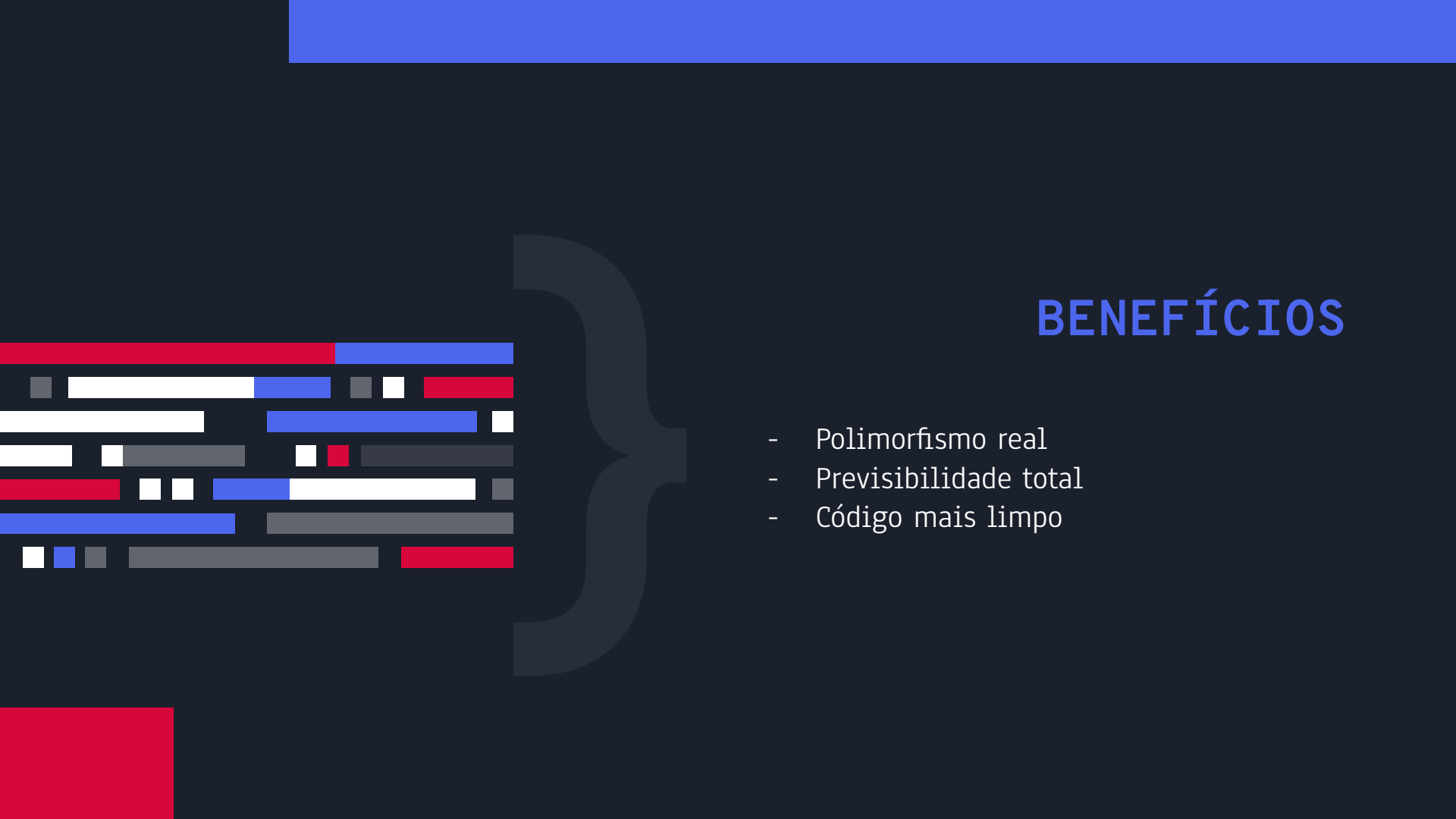
```
public class BemTeVi extends Ave implements AveVoadora {  
  
    public void voar() {  
        System.out.println("Voando até um fio de alta tensão...");  
    }  
}  
  
public class Pinguim extends Ave {}  
  
public class DocumentarioDiscovery {  
  
    public void filmarReinoAnimal(AveVoadora ave) {  
        ave.voar();  
    }  
}
```





SOLUÇÃO

A solução gira em torno de entender a diferença entre herança e reutilização automática. Herança deve ser utilizada quando queremos definir uma relação “é um” explícita, e essa relação exige que os comportamentos da subclasse sejam compatíveis com a classe ancestral.



BENEFÍCIOS

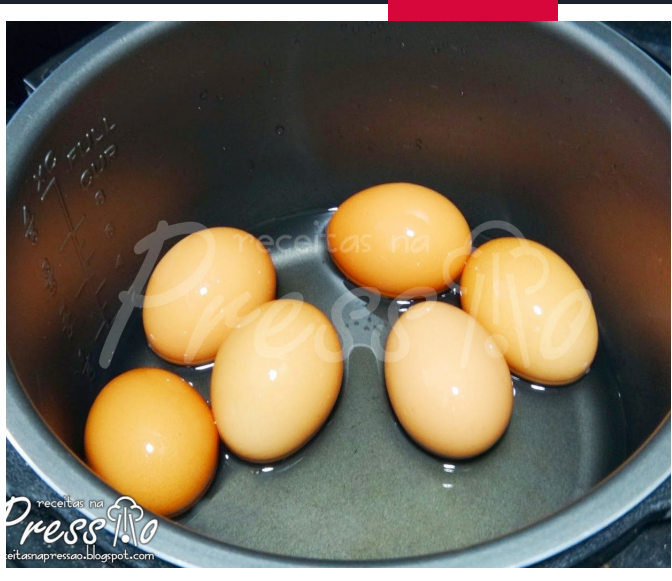
- Polimorfismo real
- Previsibilidade total
- Código mais limpo



HORA DE EXERCÍCIO

Modele um sistema de gestão de RH, onde informações sobre funcionários, tais como salários, benefícios, férias etc. Nessa empresa, existem contratações na modalidade CLT e na modalidade freelancer. Todos os funcionários da empresa possuem benefícios previstos, exceto funcionários freelancer.

Implemente esse sistema.



SEGREGAÇÃO DE INTERFACE

Conceito: Clientes não devem depender de métodos que não utilizam.

O princípio defende a criação de interfaces pequenas, específicas e focadas em um único contexto de negócio, em vez de criar interfaces genéricas e extensas que acumulam múltiplos comportamentos.



CENÁRIO

Quando criamos interfaces excessivamente grandes e genéricas (interfaces “gordas”), o design de código sofre consequências negativas:

- Dependências inúteis
- Rigidez no design
- Desperdício de recursos

PROBLEMAS

- **Dependências inúteis:** Classes implementam métodos vazios ou lançam exceções apenas para satisfazer o contrato
- **Rigidez no design:** Qualquer alteração em um método de interface obriga a atualização de todas as classes herdadas
- **Desperdício de recursos:** Módulos clientes acabam importando e compilando lógicas de que não precisam para funcionar

EXEMPLO

```
public class ImpressoraTermica implements Multifuncional {  
  
    public void imprimir() {  
        System.out.println("Imprimindo cupom...");  
    }  
  
    public void escanear() {  
        throw new UnsupportedOperationException();  
    }  
  
    public void copiar() {  
        throw new UnsupportedOperationException();  
    }  
}
```

```
public interface Multifuncional {  
  
    void imprimir();  
    void escanear();  
    void copiar();  
  
}
```

Quais problemas podem ser identificados nessa classe?





PROBLEMA

Padronizar as operações possíveis de uma impressora até parece uma boa ideia, mas acaba se tornando um problema quando vamos para cenários específicos, como a impressora térmica, que só imprime. Todos os outros métodos se tornam inúteis, ferindo o princípio.

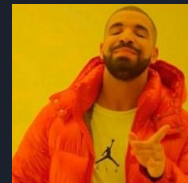
EXEMPLO

```
public interface Impressora {  
  
    void imprimir();  
  
}  
  
public interface Escaner {  
  
    void escanear();  
  
}  
  
public interface Copiadora {  
  
    void copiar();  
  
}
```



EXEMPLO

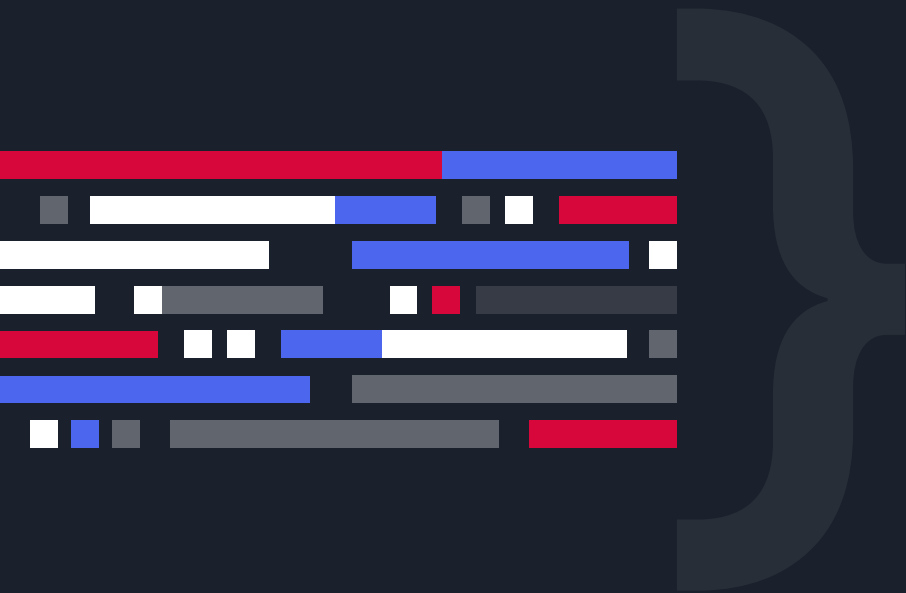
```
public class ImpressoraTermica implements Impressora {  
  
    public void imprimir() {  
        System.out.println("Imprimindo o cupom...");  
    }  
}
```



EXEMPLO

```
public class ImpressoraMultifuncional implements Impressora, Escaner, Copiadora {  
  
    public void imprimir() {  
        System.out.println("Imprimindo o documento...");  
    }  
  
    public void escanear() {  
        System.out.println("Escaneando o documento...");  
    }  
  
    public void copiar() {  
        System.out.println("Tirando uma xerox...");  
    }  
}
```





SOLUÇÃO

Ao invés de termos uma interface grande com todas as operações possíveis, criamos interfaces menores para operações específicas. Agora, cada tipo de impressora implementa as operações que ela será capaz de dar suporte: a impressora térmica só imprime; a impressora multifuncional imprime, escaneia e tira cópias.



BENEFÍCIOS

- Desacoplamento eficaz
- Contratos enxutos
- Facilidade de manutenção
- Independência de operações



HORA DE EXERCÍCIO

Considere uma interface que representa as possíveis operações que um usuário pode fazer em um sistema. Inicialmente concentramos todas as operações em uma única interface, o que pode trazer alguns problemas de manutenção no futuro.

Considere refatorar essas interface para torná-la mais eficiente.

EXERCÍCIO

```
public interface UsuarioSistema {  
  
    void cadastrarUsuario();  
  
    void excluirUsuario();  
  
    void emitirRelatorio();  
  
    void aprovarPagamento();  
  
}
```



INVERSÃO DE DEPENDÊNCIA

Conceito: Módulos de alto nível não devem depender de módulos de baixo nível. Ambos devem depender de abstrações.

Neste princípio, o código estrutural e de regras de negócio deve ser blindado contra mudanças em ferramentas externas, bibliotecas e infraestrutura.



CENÁRIO

Quando classes que contém regras de negócio dependem diretamente de classes que realizam tarefas operacionais de infraestrutura, o sistema se degrada rapidamente:

- Rigidez extrema
- Inviabilidade de testes
- Arquitetura frágil



PROBLEMAS

- **Rigidez extrema:** Trocar um banco de dados ou um serviço de e-mail exige reescrever a lógica central do negócio
 - **Inviabilidade de testes:** Fica impossível testar regras de negócio de forma isolada sem disparar conexões reais com bancos ou APIs externas
 - **Arquitetura frágil:** Alterações simples em uma biblioteca de terceiro geram um efeito cascata de erros por todo o sistema
- 

EXEMPLO

```
public class PedidoService {  
  
    private MySQLPedidoRepository repository = new MySQLPedidoRepository();  
  
    public void salvar() {  
        repository.salvar();  
    }  
}
```

Quais problemas podem ser identificados nessa classe?





PROBLEMA

A classe de serviço de pedidos executa as regras de negócio para processar um pedido, e interage com um banco de dados MySQL, gerando assim um forte acoplamento. Mudar para um outro banco de dados vai exigir mudanças complexas na classe de serviço, deixando a arquitetura frágil a mudanças e ferindo o princípio.

EXEMPLO

```
public interface PedidoRepository {  
  
    void salvar();  
  
}  
  
public class MySQLPedidoRepository implements PedidoRepository {  
  
    public void salvar() {  
        System.out.println("Salvando o pedido no banco de dados MySQL...");  
    }  
  
}
```



EXEMPLO

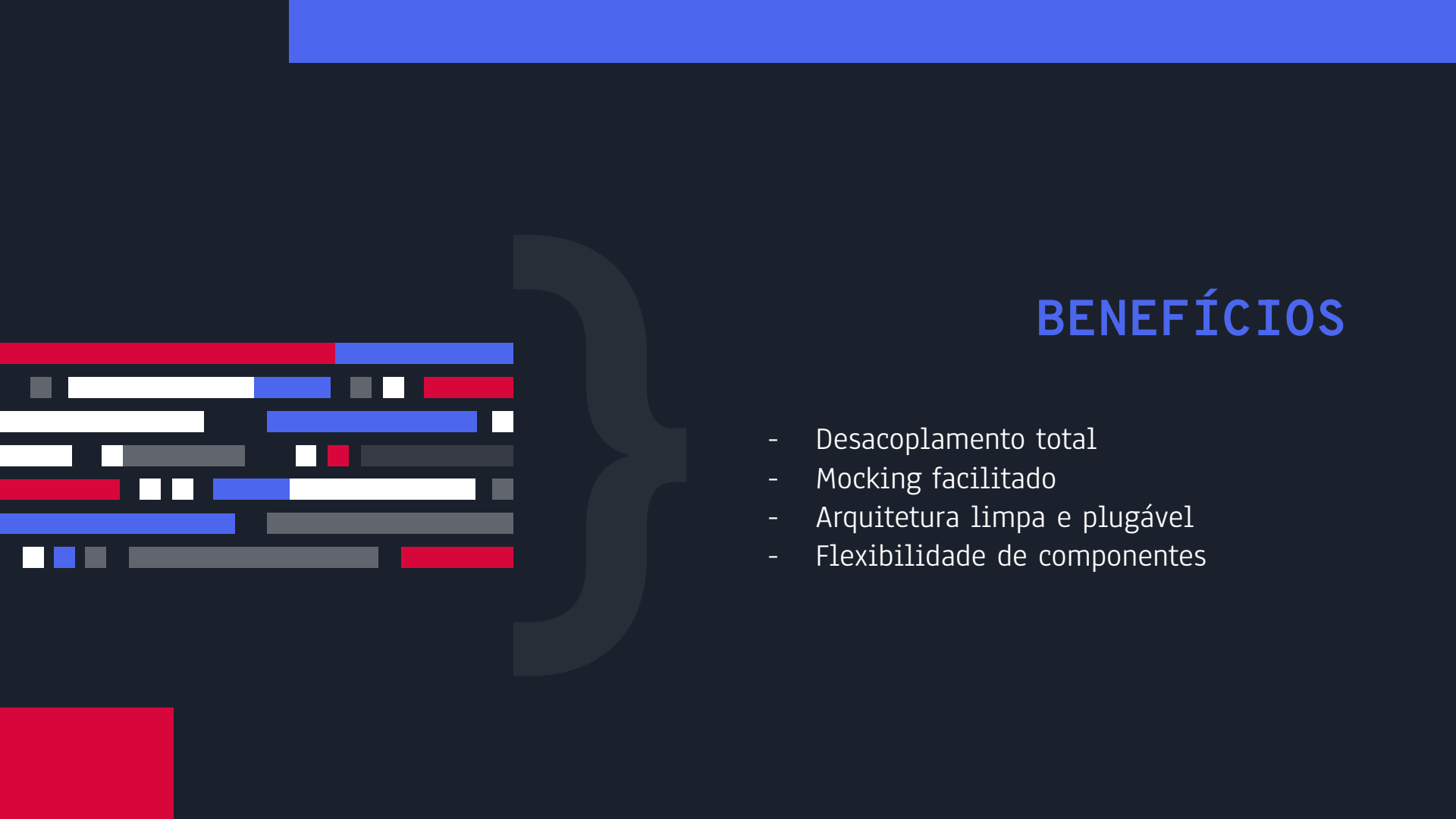
```
public class PedidoService {  
  
    private PedidoRepository repository;  
  
    public PedidoService(PedidoRepository repository) {  
        this.repository = repository;  
    }  
  
    public void salvar() {  
        repository.salvar();  
    }  
}
```





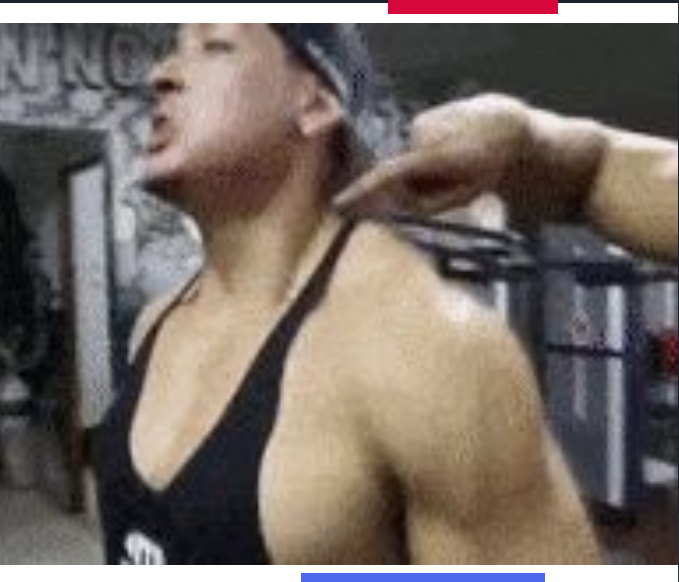
SOLUÇÃO

O primeiro ponto de modificação é a definição de uma abstração para as operações de banco de dados, onde cada serviço específico deve as implementar. Na classe de serviço, recebemos uma instância dessas operações criadas externamente, tornando componente independente de implementação específica.



BENEFÍCIOS

- Desacoplamento total
- Mocking facilitado
- Arquitetura limpa e plugável
- Flexibilidade de componentes



HORA DE EXERCÍCIO

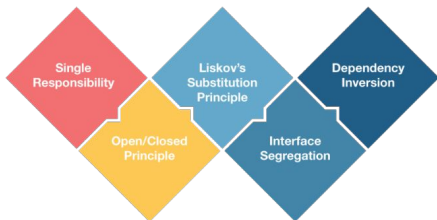
Vamos implementar um serviço de envio de mensagens multicanal. Ao digitar uma mensagem para envio, podemos escolher em qual meio de comunicação ela será disparada (e-mail, SMS, Whatsapp etc). O ideal é que essa definição do meio de comunicação seja flexível.

Implemente esse serviço.

CONCLUSÃO

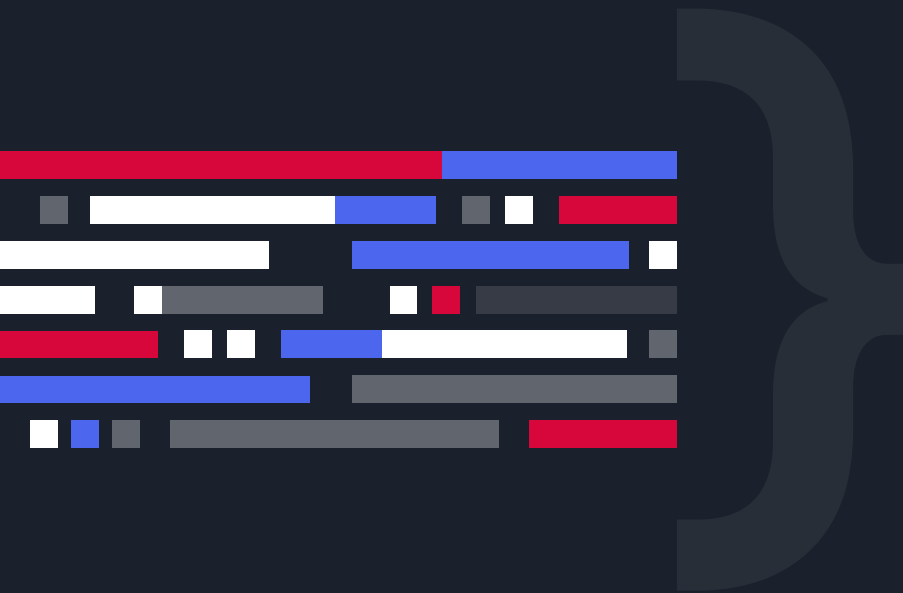


S.O.L.I.D.



CUIDADOS

Os princípios SOLID vêm para nos ajudar no design de aplicações e construirmos sistemas mais robustos e eficientes. Porém, cada análise de cada aplicação se torna única e necessária, para evitarmos de adicionar complexidade desnecessária. Excesso de abstrações não tornam um sistema robusto, podendo até mesmo trazer os mesmos problemas que devemos fugir.

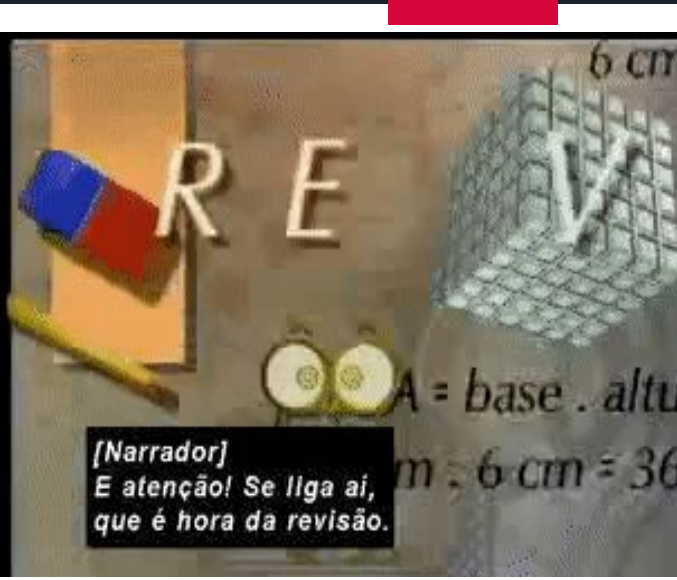


EQUILÍBRIO

Equilibrar a complexidade com as necessidades arquiteturais é o que faz um bom projetista de sistemas orientados a objetos. Adquirimos esse equilíbrio fazendo a leitura adequada de cada cenário, experimentando diferentes soluções e encontrando aquilo que há de melhor em cada uma e que se encaixa no problema.

PRINCÍPIOS

Princípio	Objetivo
Responsabilidade Única	Responsabilidades bem definidas para classes
Aberto/Fechado	Estender comportamentos sem modificar o original
Substituição de Liskov	Substituição correta de objetos herdados
Segregação de Interface	Definição de interfaces específicas para cada cenário
Inversão de Dependência	Depender de abstrações, ao invés de componentes específicos



RECAPITULANDO

- Histórico
- Conceito principal
- Princípio da Responsabilidade Única
- Princípio Aberto/Fechado
- Princípio da Substituição de Liskov
- Princípio da Segregação de Interface
- Princípio da Inversão de Dependência

OBRIGADO!



"- ESTÁ COMENDO FEITO UMA MACACA, TEM QUE MASTIGAR A COMIDA!
- MAS É SOPA!"

As Aventuras de Jackie Chan